



CRC 2026

#5 Wzorce Projektowe

ING Hubs Poland Bartosz Gawron

April 2026



Agenda

- Zasady spotkania
 - Przerwa (15min w połowie zajęć ?)
- Przedstawmy się
- Inżyniera oprogramowania
- Wzorce projektowe
 - Singleton
 - Strategy
 - Factory
 - Builder
 - Façade
 - Decorator
 - Adapter (np. JDBC + JPA)
- Podsumowanie
- <https://github.com/SirBarto/CRC-2026-Demo> <-- tu znajdziesz praktyczne przykłady z zajęć

Przedstawmy się



Bartosz Gawron

- bartosz.gawron@ing.com



Wstęp

Inżynieria oprogramowania

IO

- **Inżynier oprogramowania** to specjalista zajmujący się projektowaniem, tworzeniem, testowaniem i utrzymywaniem systemów informatycznych. Jego praca opiera się na stosowaniu zasad inżynierii (metodycznego podejścia) do procesu tworzenia oprogramowania.

Kluczowe umiejętności

znajomość języków programowania (np. Java 17)

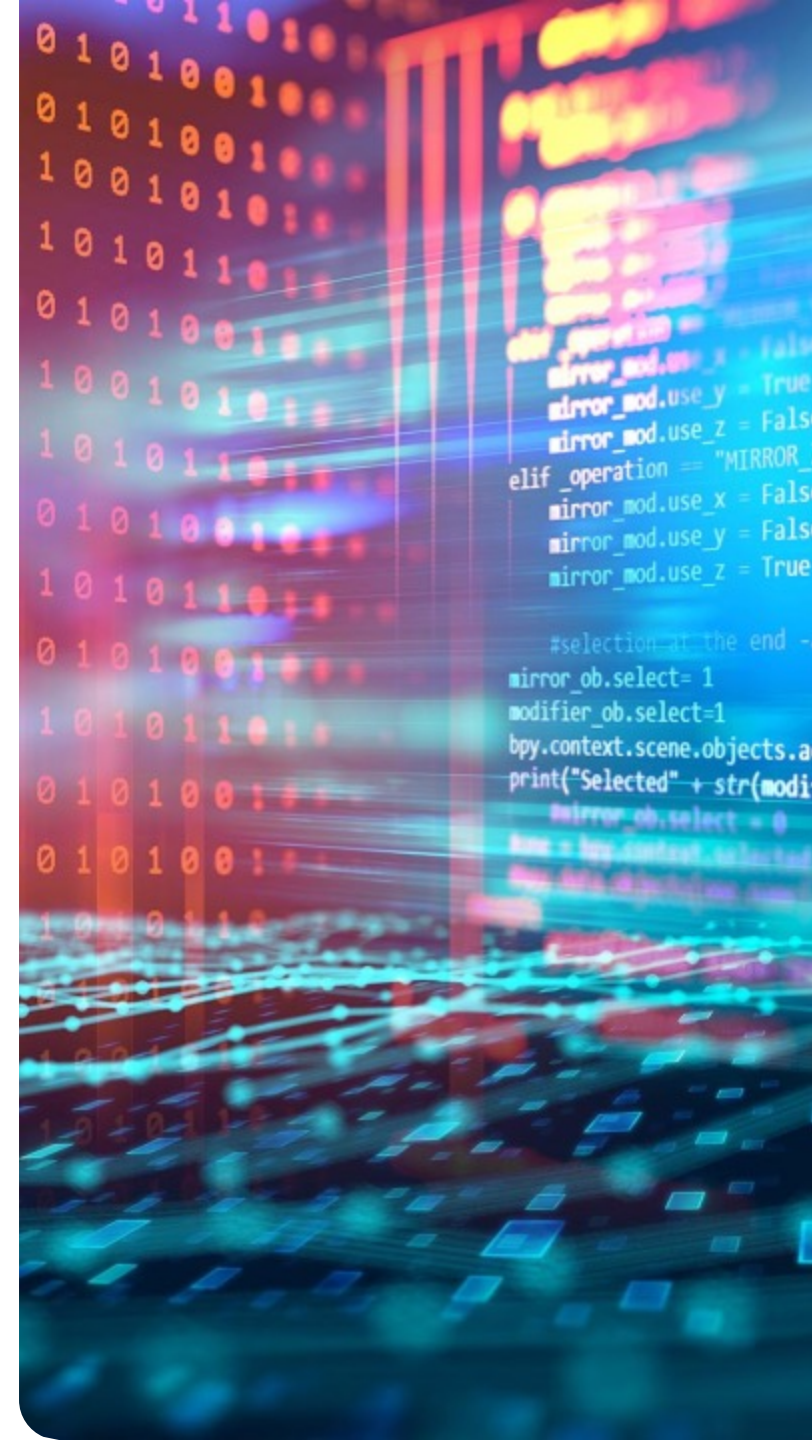
znajomość wzorców projektowych (Strategy, Factory, Singleton itd.)

umiejętność pracy z systemami kontroli wersji (Git)

znajomość zasad **SOLID** i dobrych praktyk

testowanie (JUnit, Mockito)

podstawy baz danych i SQL



Wzorce projektowe

```
def operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
elif operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

and -add back the deselected mirror modifier

```
mirror_ob.select=1  
modifier_ob.select=1  
bpy.context.scene.objects.active = modifier_ob  
print("Selected" + str(modifier_ob)) # modifier ob is the active ob  
mirror_ob.select = 0  
name = bpy.context.selected_objects[0]  
name.data.object.name = name.name + " - Mirror"
```


Troszkę teorii (...)

“Wzorzec projektowy
- to zbiór najlepszych
praktyk z gotowymi
rozwiązaniami dla
wybranych
problemów
napotykanym w
trakcie projektowania
rozwiązań
zorientowanych
obiektowo.”
P.Bykowski



Z czego składa się wzorzec projektowy?

nazwy

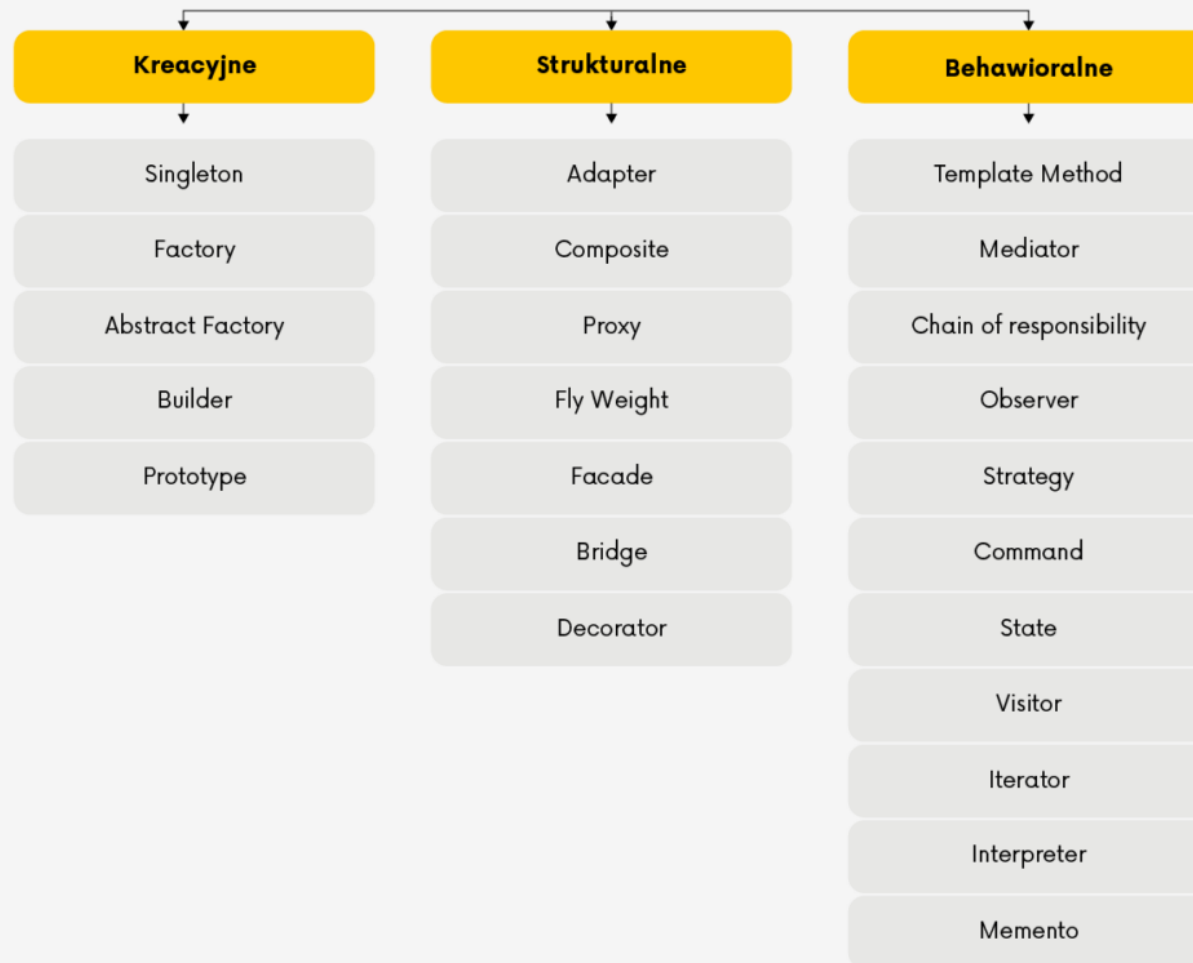
opisu problemu, który rozwiązuje

rozwiązania

konsekwencji jego użytkowania

Klasyfikacja wzorców projektowych

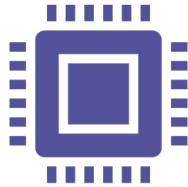
Kategorie wzorców projektowych w Java



Korzyści płynące ze stosowania wzorców projektowych



Spójność z koncepcją SOLID



Clean code – lepsza czytelność i zrozumiałość kodu



Łatwość komunikacji - znacznie prościej jest przekazać koledze nazwę wzorca jaki ma zaimplementować niż starać się to wyjaśnić opisowo.



Ułatwienie testowania

Implementacja wybranych wzorców projektowych



Strategy



Factory



Builder



Singleton



Facade



Decorator



Adapter

Singleton

Idea

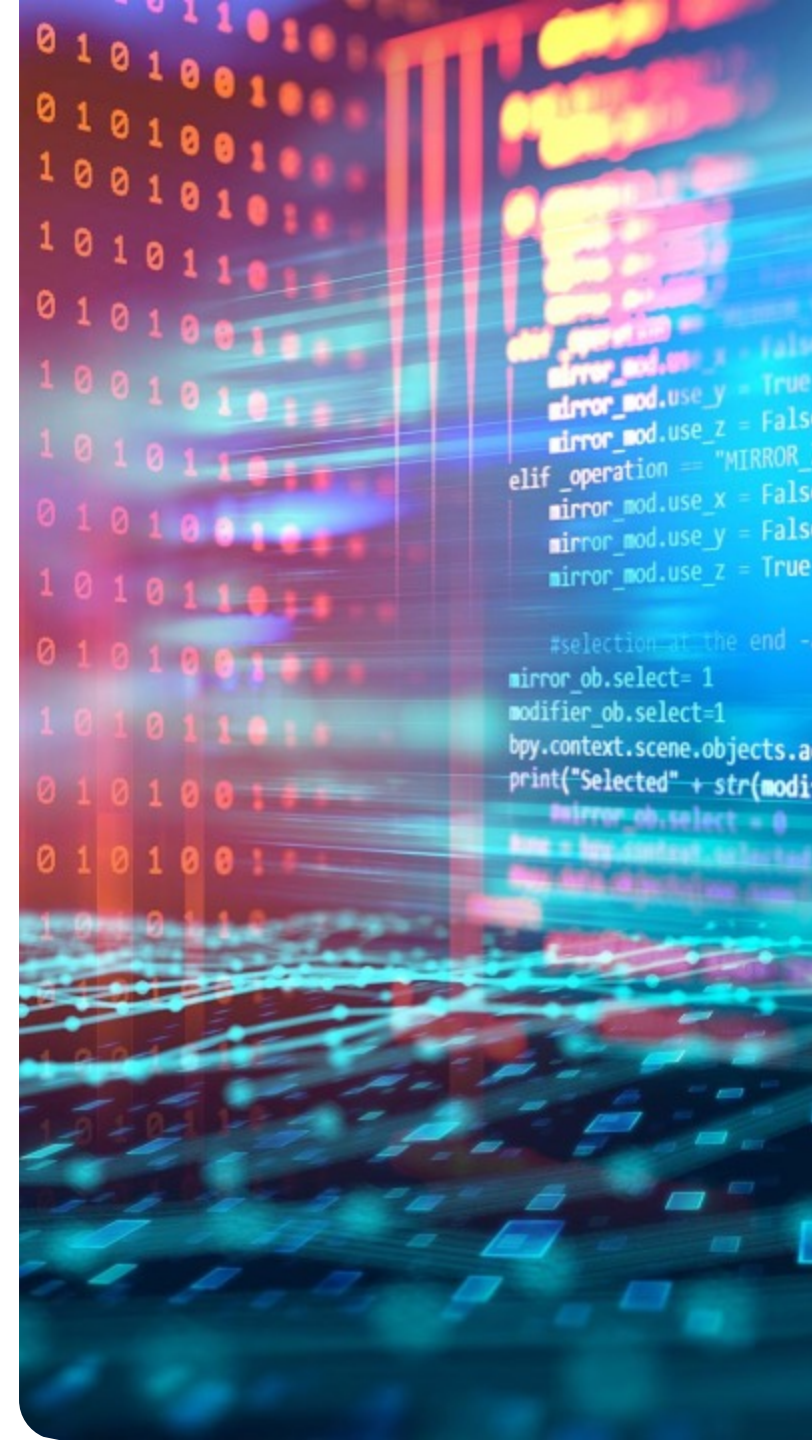


- ☐ **prywatny konstruktor**, który uniemożliwia tworzenie obiektów z zewnątrz,
- ☐ **statyczne pole**, przechowujące jedną instancję,
- ☐ **publiczną metodę statyczną**, zwracającą tę instancję.

Struktura



- ☐ przechowuje jedną instancję,
- ☐ kontroluje jej tworzenie,
- ☐ udostępnia globalny punkt dostępu (np. getInstance()).



Singleton

Zalety

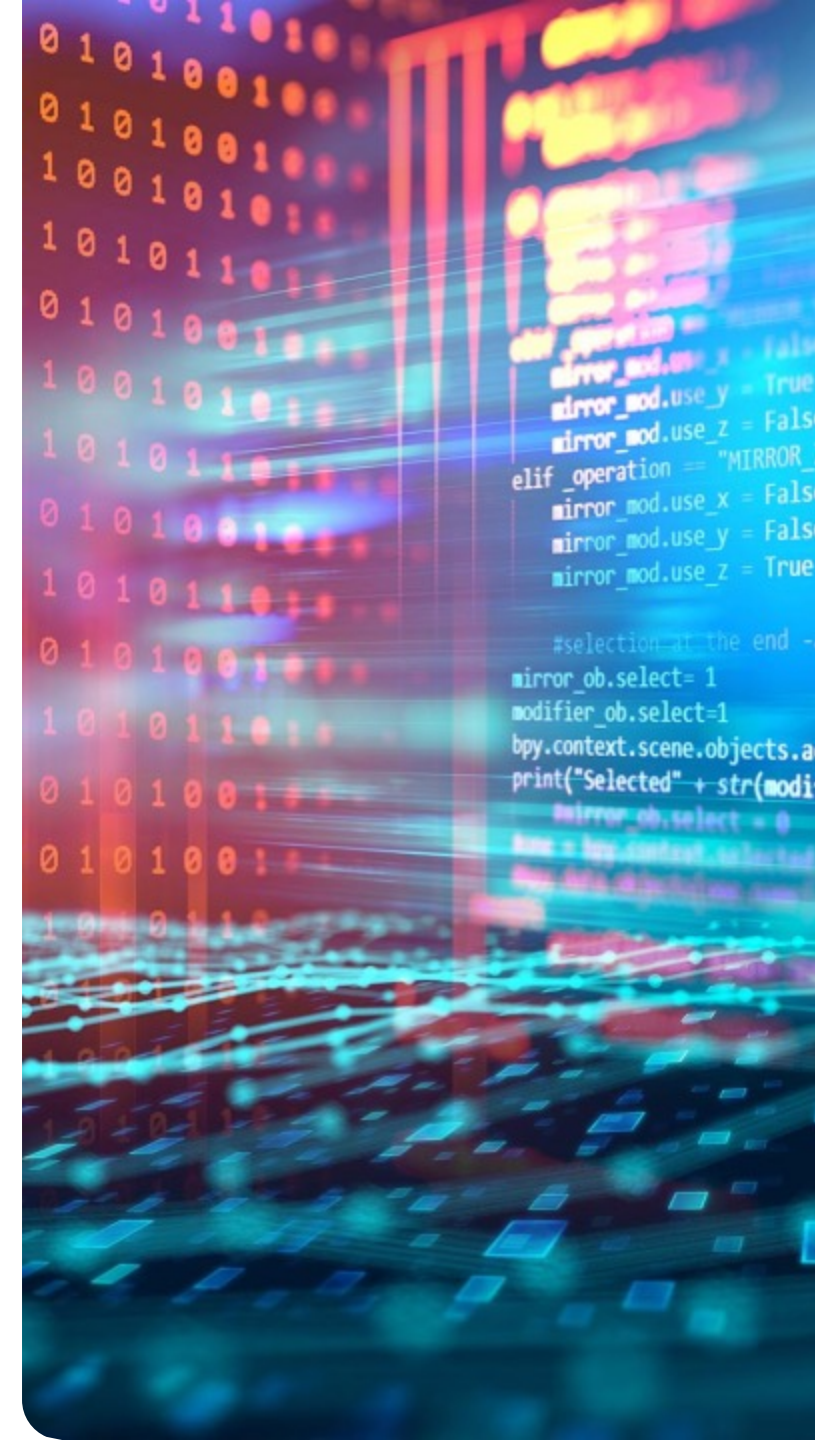
- **Kontrola liczby instancji (dokładnie jedna)**
- **Globalny dostęp do obiektu**
- Oszczędność zasobów (np. połączenia, konfiguracje)
- Przydatny dla współdzielonych zasobów

Wady

- **Łamie zasadę pojedynczej odpowiedzialności (SRP)**
- Utrudnia testowanie (trudne mockowanie)
- Wprowadza globalny stan (może prowadzić do błędów)
- Problemy w środowiskach wielowątkowych (wymaga synchronizacji)

Przykłady zastosowania

- zarządzanie konfiguracją aplikacji,
- logowanie (logger),
- zarządzanie połączeniem z bazą danych,
- cache lub menedżery zasobów.



Strategy (strategia)

Idea

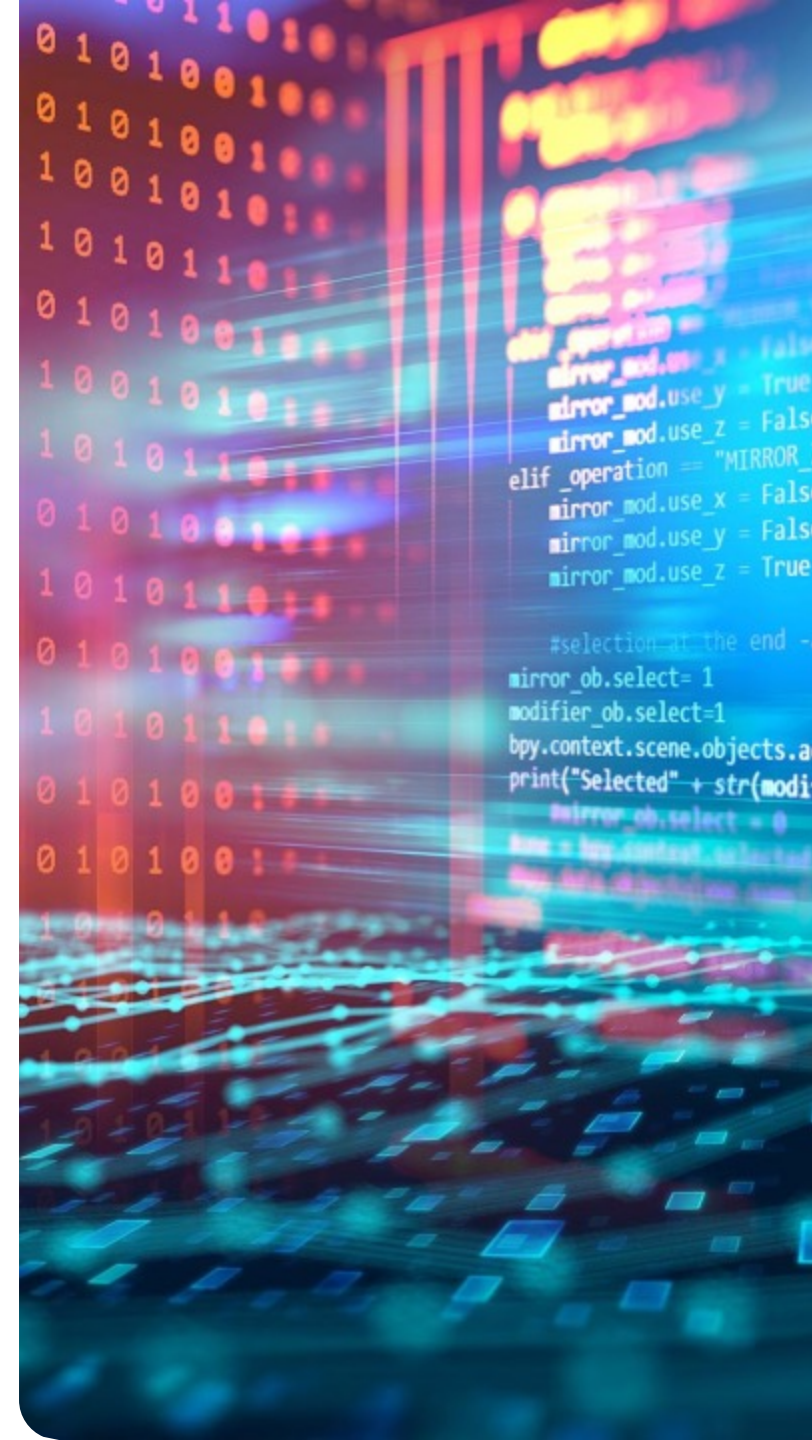


- ☐ Enkapsulacja różnych algorytmów w osobnych klasach
- ☐ Możliwość ich dynamicznej zmiany w trakcie działania programu
- ☐ Eliminacja rozbudowanych instrukcji if/switch

Struktura



- ☐ Strategia (interfejs) – deklaruje wspólny dla wszystkich implementacji algorytmu
- ☐ Konkretny Strateg – impl interfejs strategii, definiując konkretną wersję algorytmu
- ☐ Kontekst – klasa korzystająca ze strategii przechowuje referencję do obiektu strategii i deleguje do niego wykonanie operacji



Strategy (strategia)

Zalety

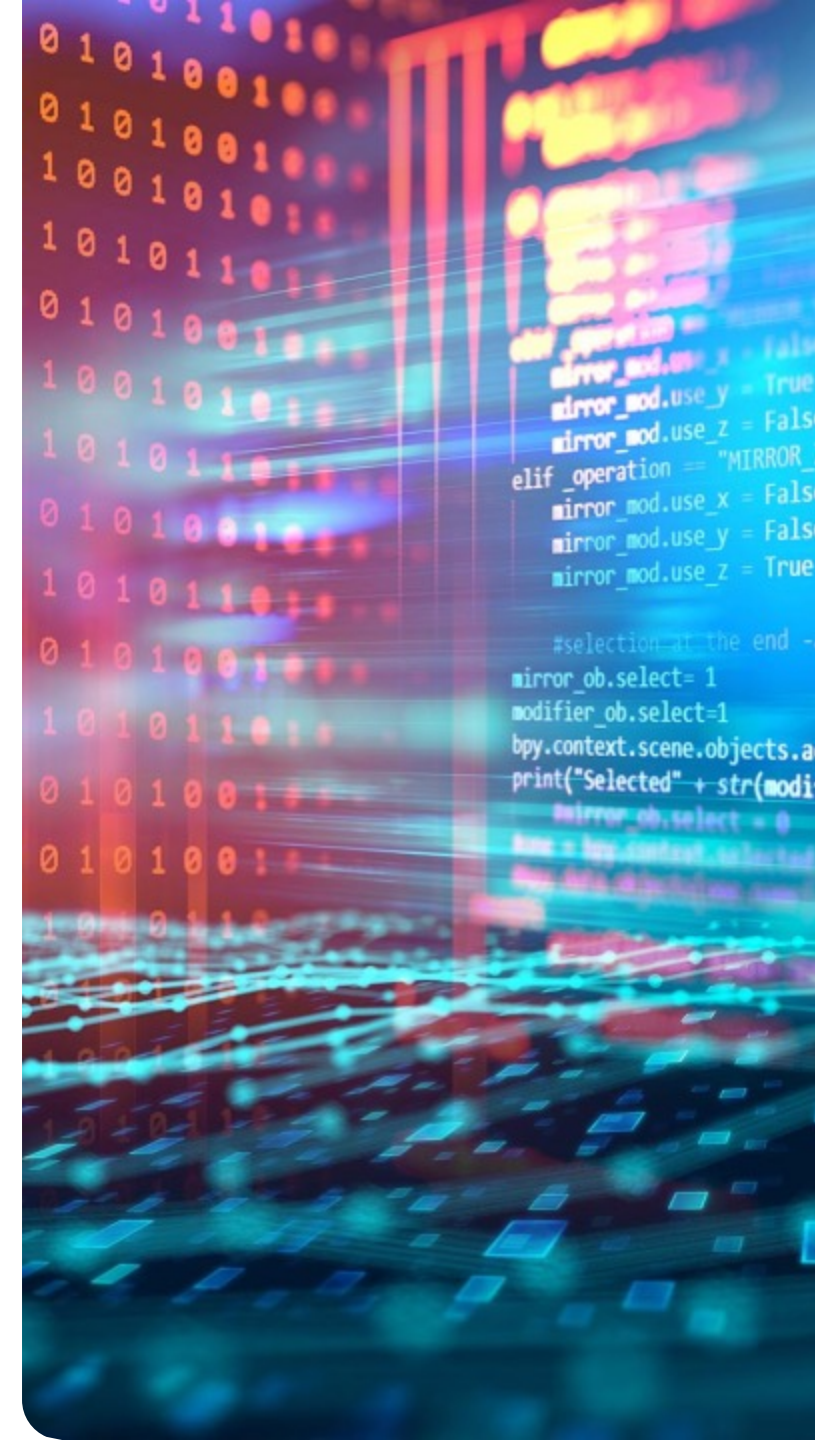
- Enkapsulacja różnych algorytmów w osobnych klasach
- Możliwość ich dynamicznej zmiany w trakcie działania programu
- Eliminacja rozbudowanych instrukcji if/switch

Wady

- Strategia (interfejs) – deklaruje wspólny dla wszystkich implementacji algorytmu
- Konkretny Strateg – impl interfejs strategii, definiując konkretną wersję algorytmu
- Kontekst – klasa korzystająca ze strategii przechowuje referencję do obiektu strategii i deleguje do niego wykonanie operacji

Przykłady zastosowania

- Różne metody sortowania
- Strategia płatności (karta kredytowa, PayPal, przelew)
- Różne metody sortowania
- Alg kompresji danych
- Różne sposoby walidacji lub przetwarzania danych



Factory (fabryka)

Idea

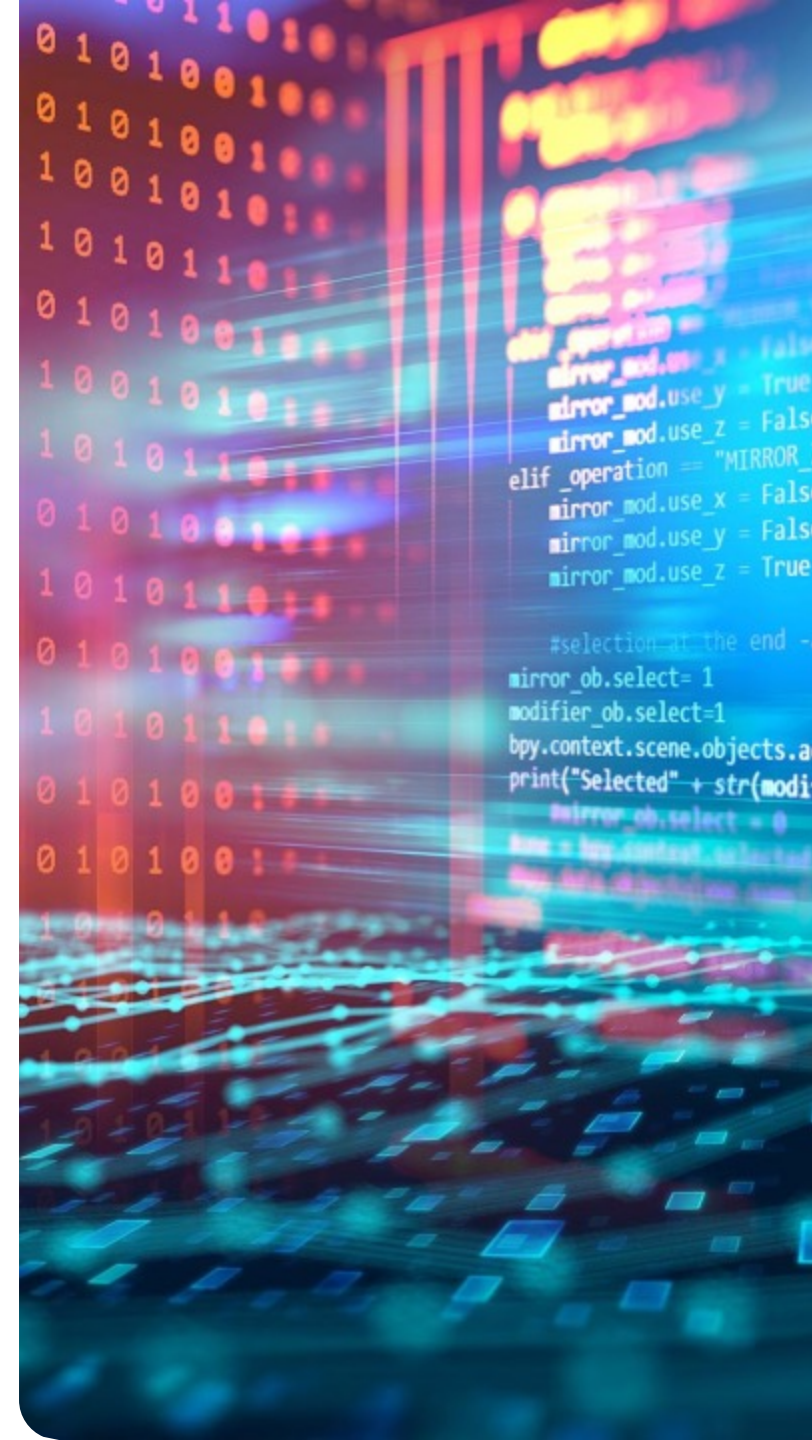


- ☐ Ukrywa szczegóły tworzenia obiektów
- ☐ Zmniejsza zależności między klasami
- ☐ Łatwo podmienia konkretne implementacje

Struktura



- ☐ Produkt – Interfejs lub klasa abstrakcyjna określająca wspólne zachowanie tworzonych obiektów
- ☐ Konkretny produkt – rzeczywiste implementacje produktu
- ☐ Twórca – klasa deklarująca metodę wytwórczą, która zwraca obiekt typu Product.
- ☐ Konkretny twórca – implementuje metodę wytwórczą, decydując, jaki obiekt zostanie utworzony



Factory (fabryka)

Zalety

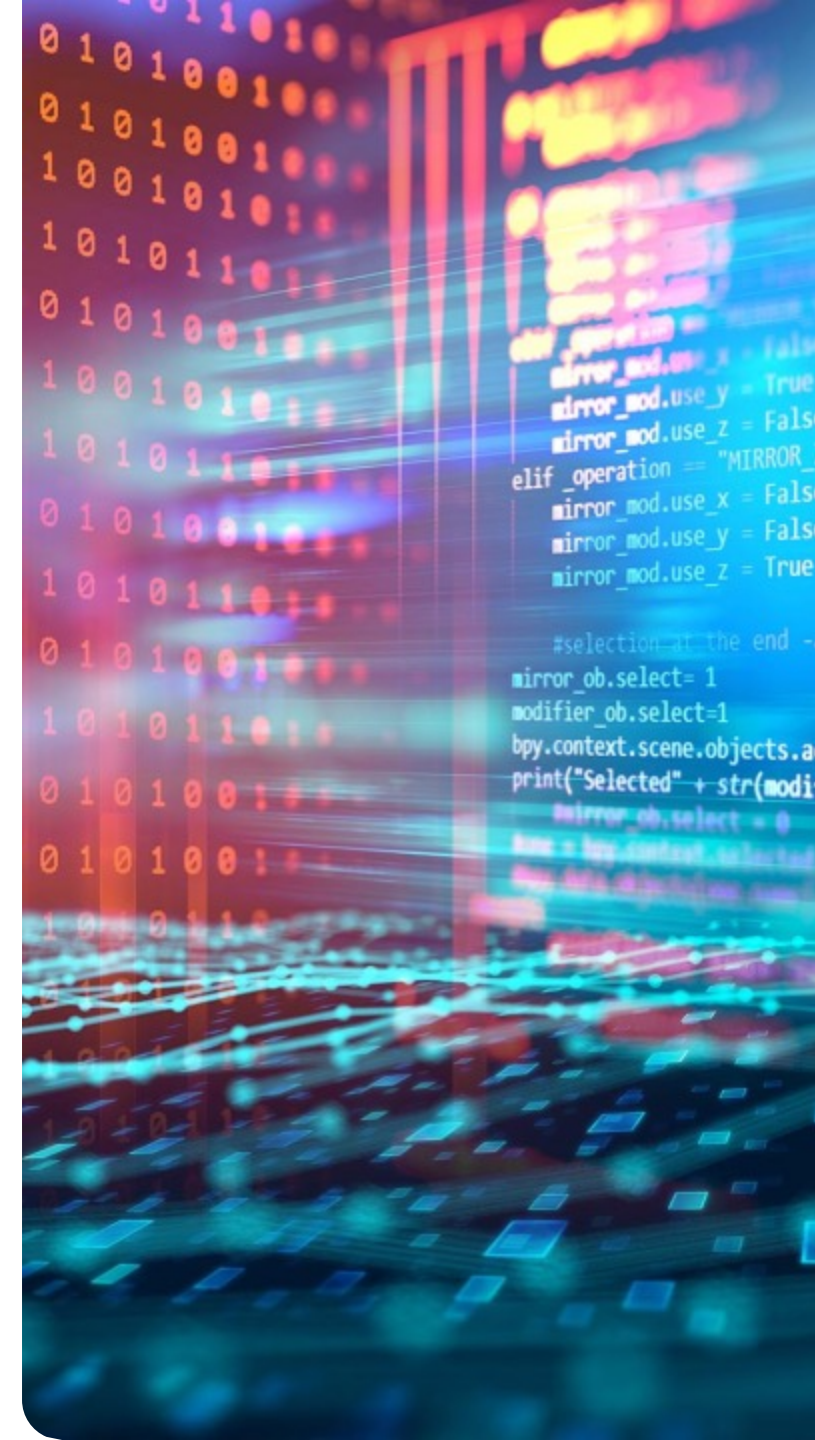
- **Luźne powiązania (low coupling)** między klasami
- **Łatwa rozbudowa** o nowe typy obiektów
- Ukrycie logiki tworzenia obiektów
- Zgodność z zasadą Open/Closed

Wady

- **Większa liczba klas i abstrakcji**
- Może wprowadzać niepotrzebną złożoność w prostych przypadkach
- Konieczność tworzenia dodatkowych klas twórców

Przykłady zastosowania

- systemy wymagające tworzenia wielu wariantów obiektów,
- biblioteki i frameworki,
- sytuacje, gdy typ obiektu jest określany dynamicznie (np. na podstawie konfiguracji).



Builder (budowniczy)

Idea

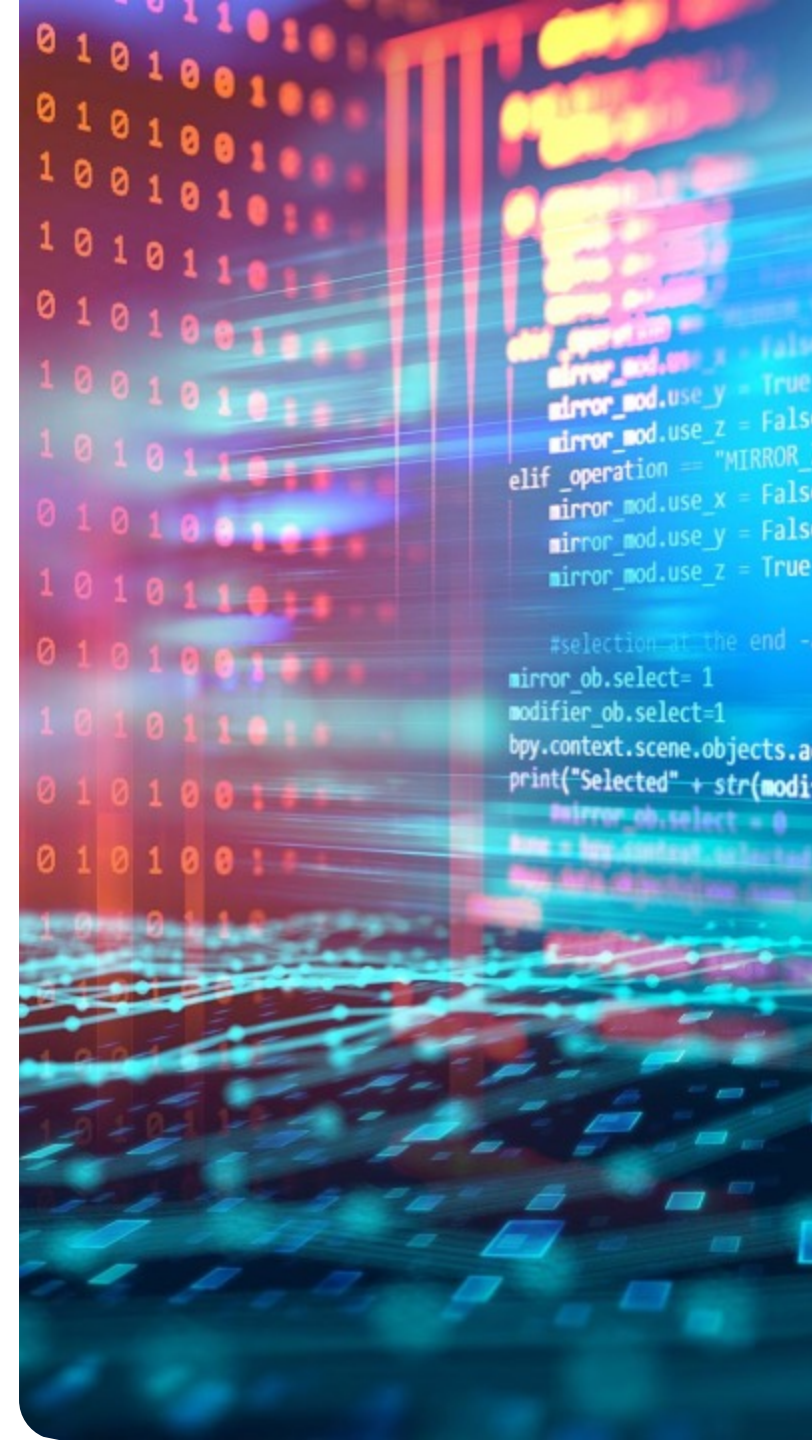


- ☐ obiekt ma wiele opcjonalnych parametrów,
- ☐ proces tworzenia jest złożony i wymaga wielu etapów,
- ☐ chcemy uniknąć tzw. „telescoping constructor” (wielu przecięzionych konstruktorów).

Struktura



-  **Builder (interfejs lub klasa abstrakcyjna)**
Definiuje metody budowania poszczególnych części obiektu.
-  **Concrete Builder (konkretny budowniczy)**
Implementuje interfejs Builder i tworzy konkretną reprezentację obiektu.
-  **Produkt (Product)**
Złożony obiekt, który jest budowany.
-  **Dyrektor (Director) (opcjonalny)**
Zarządza procesem budowy, definiując kolejność wywoływania metod budowniczego.



Builder (budowniczy)

Zalety

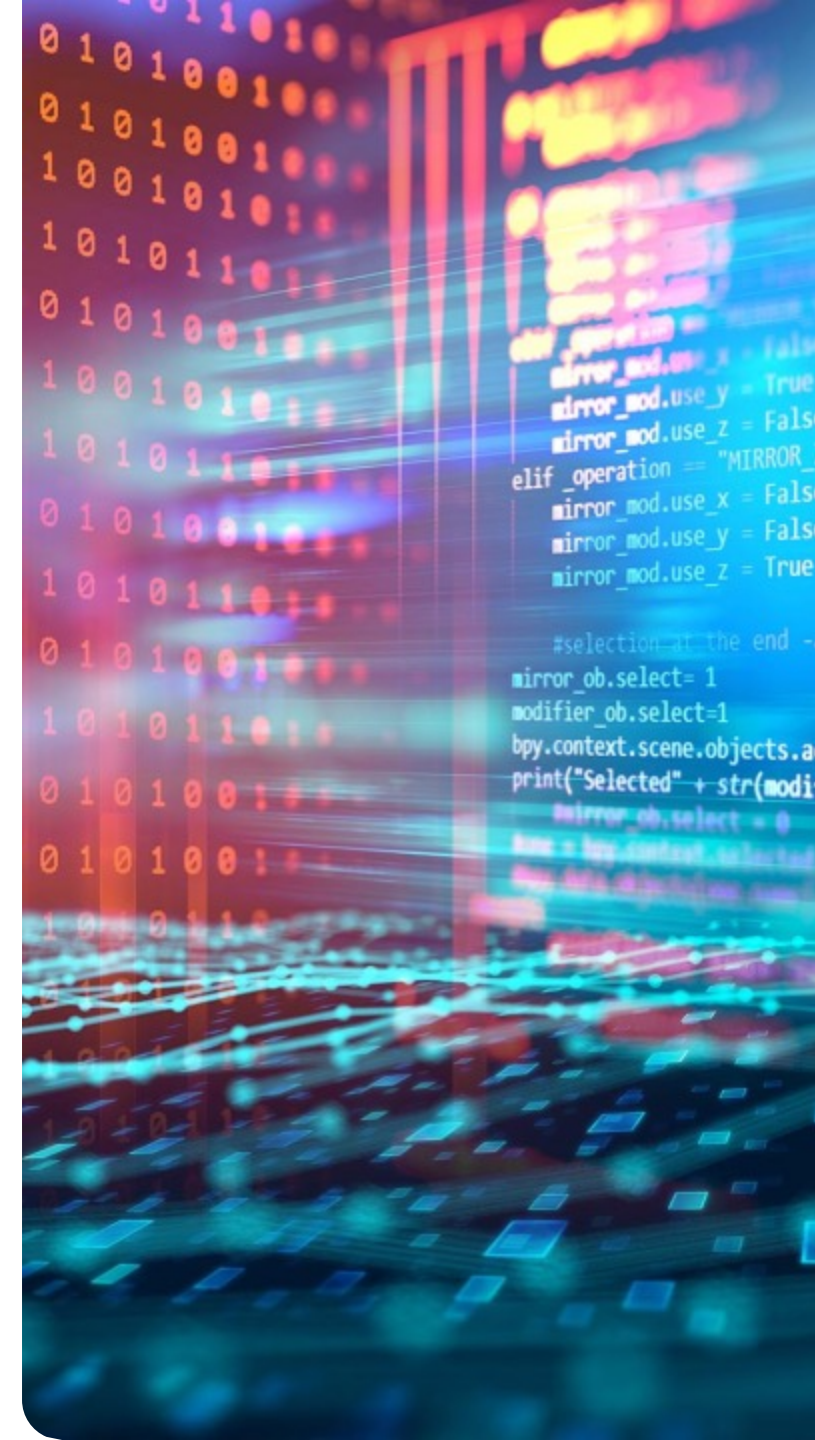
- **Luźne powiązania (low coupling)** między klasami
- **Łatwa rozbudowa** o nowe typy obiektów
- Ukrycie logiki tworzenia obiektów
- Zgodność z zasadą Open/Closed

Wady

- **Większa liczba klas i abstrakcji**
- Może wprowadzać niepotrzebną złożoność w prostych przypadkach
- Konieczność tworzenia dodatkowych klas twórców

Przykłady zastosowania

- systemy wymagające tworzenia wielu wariantów obiektów,
- biblioteki i frameworki,
- sytuacje, gdy typ obiektu jest określany dynamicznie (np. na podstawie konfiguracji).



Façade (fasada)

Idea

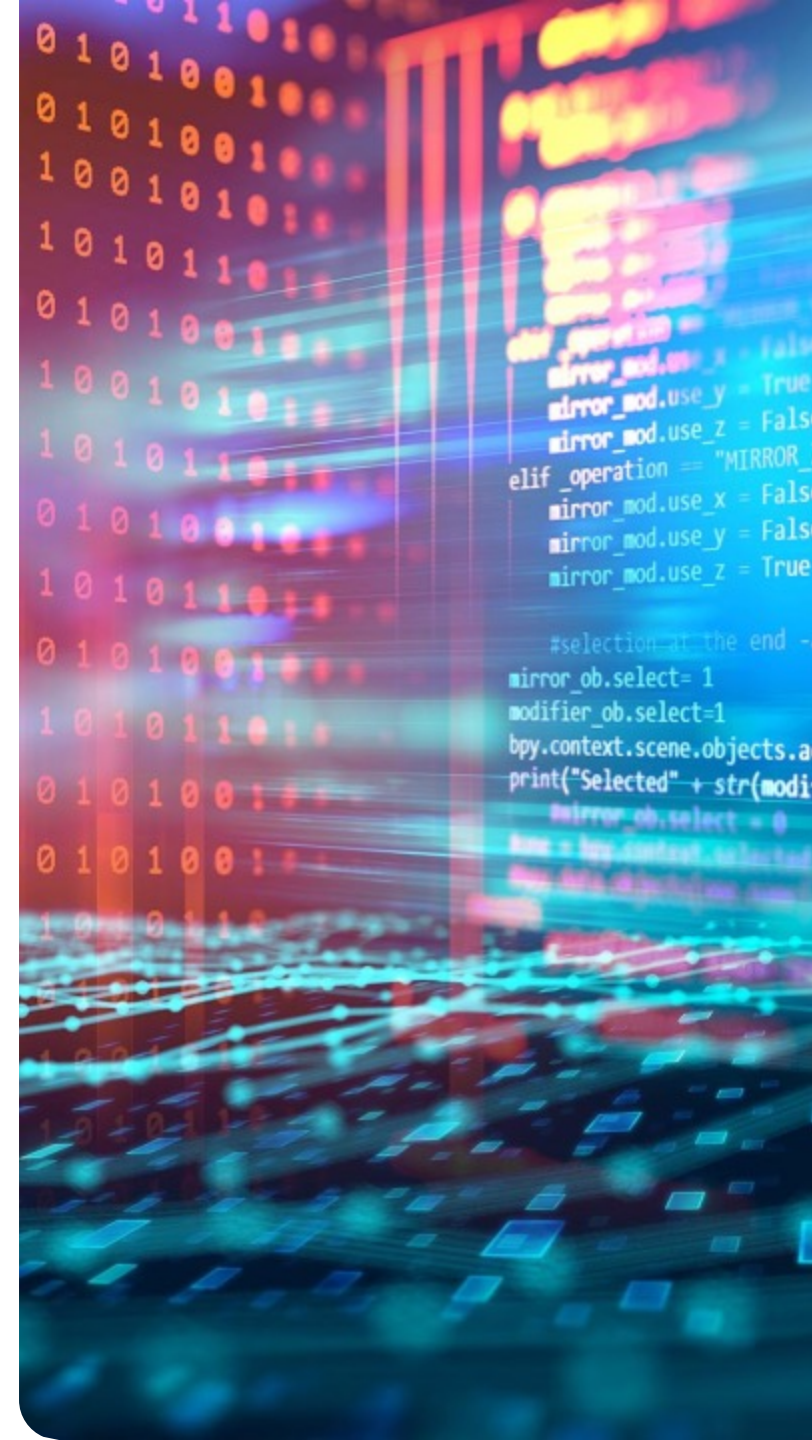


- ☐ zmniejszyć stopień skomplikowania kodu po stronie klienta,
- ☐ ograniczyć zależności między komponentami,
- ☐ ukryć szczegóły implementacyjne systemu.

Struktura



- ☐ **Fasada (Facade)**
Udostępnia uproszczony interfejs do złożonego podsystemu i deleguje wywołania do odpowiednich klas.
- ☐ **Podsystem (Subsystem classes)**
Zbiór klas realizujących właściwą logikę aplikacji. Nie są świadome istnienia fasady.
- ☐ **Klient (Client)**
Korzysta z fasady zamiast bezpośrednio z klas podsystemu.



Façade (fasada)

Zalety

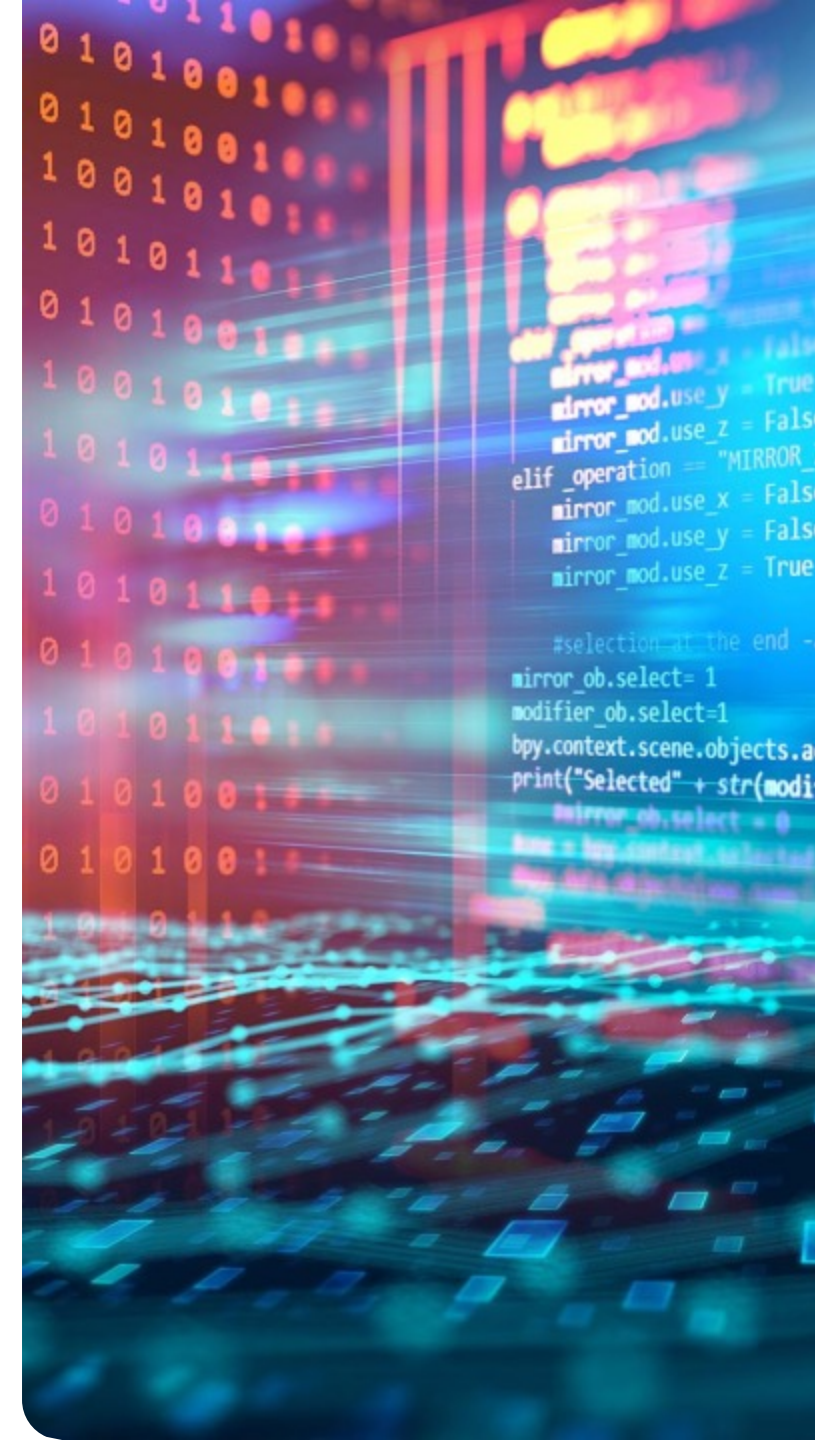
- **Uproszczenie interfejsu systemu**
- **Zmniejszenie zależności (low coupling)** między klientem a podsystemem
- **Lepsza czytelność i organizacja kodu**
- Ułatwienie pracy z rozbudowanymi systemami

Wady

- Możliwość powstania „klasy Boga” (nadmiernie rozbudowanej fasady)
- Ograniczenie dostępu do zaawansowanych funkcji podsystemu
- Dodatkowa warstwa abstrakcji

Przykłady zastosowania

- uproszczenie pracy z bibliotekami lub frameworkami,
- integracja złożonych systemów (np. API),
- systemy o wielu zależnych komponentach,
- tworzenie warstwy usługowej w aplikacjach.



Decorator (dekorator)

Idea

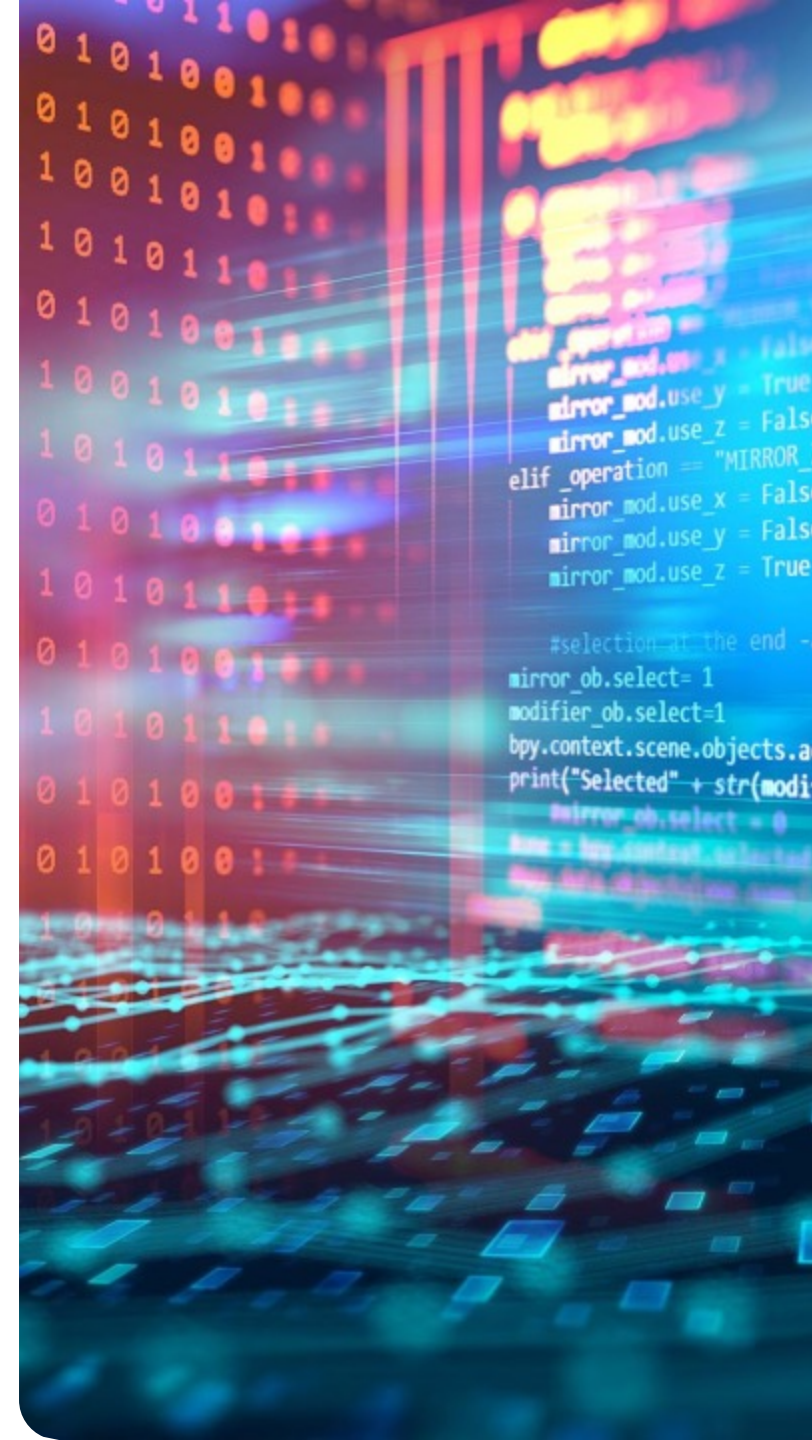


- ☐ implementuje ten sam interfejs co obiekt bazowy,
- ☐ przechowuje referencję do oryginalnego obiektu,
- ☐ rozszerza jego funkcjonalność, delegując wywołania i dodając własne zachowanie.

Struktura



- ☐ **Komponent (Component)**
Interfejs definiujący wspólne operacje.
- ☐ **Konkretny komponent (Concrete Component)**
Bazowa implementacja obiektu.
- ☐ **Dekorator (Decorator)**
Klasa abstrakcyjna implementująca interfejs komponentu i przechowująca referencję do komponentu.
- ☐ **Konkretny dekorator (Concrete Decorator)**
Rozszerza funkcjonalność komponentu poprzez dodanie nowych zachowań przed lub po wywołaniu oryginalnej metody.



Decorator (dekorator)

Zalety

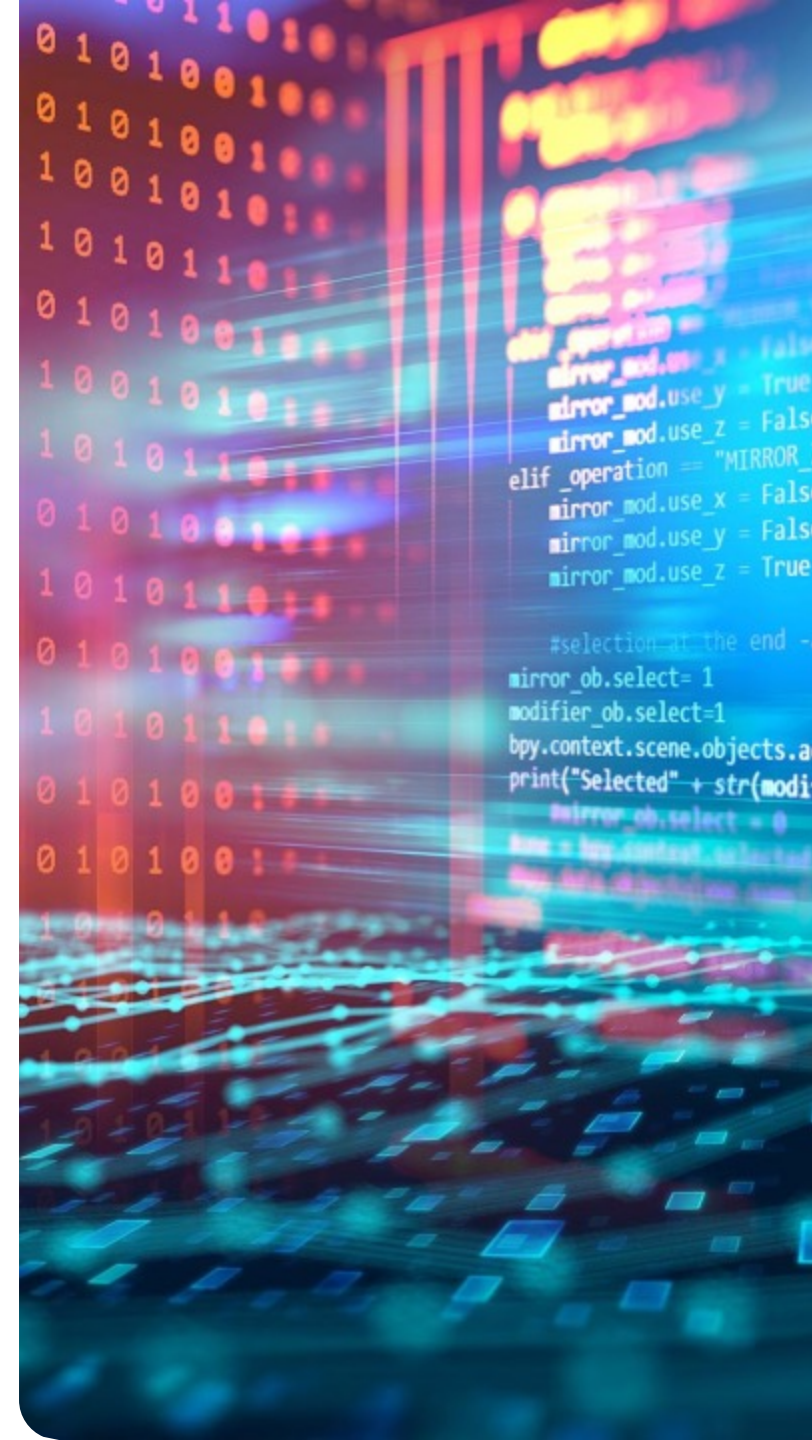
- **Dynamiczne rozszerzanie funkcjonalności**
- Eliminacja rozbudowanej hierarchii dziedziczenia
- **Zgodność z zasadą Open/Closed**
- Możliwość łączenia wielu dekoratorów

Wady

- **Zwiększona liczba małych klas**
- Trudniejsza analiza przepływu (wiele warstw dekoratorów)
- Konieczność uwzględnienia kolejności dekoratorów

Przykłady zastosowania

- dodawanie funkcji do obiektów w runtime (np. logowanie, szyfrowanie),
- systemy GUI (rozszerzanie komponentów wizualnych),
- strumienie danych (np. biblioteki I/O),
- sytuacje, gdzie wymagane są różne kombinacje funkcjonalności.



Adapter

Idea

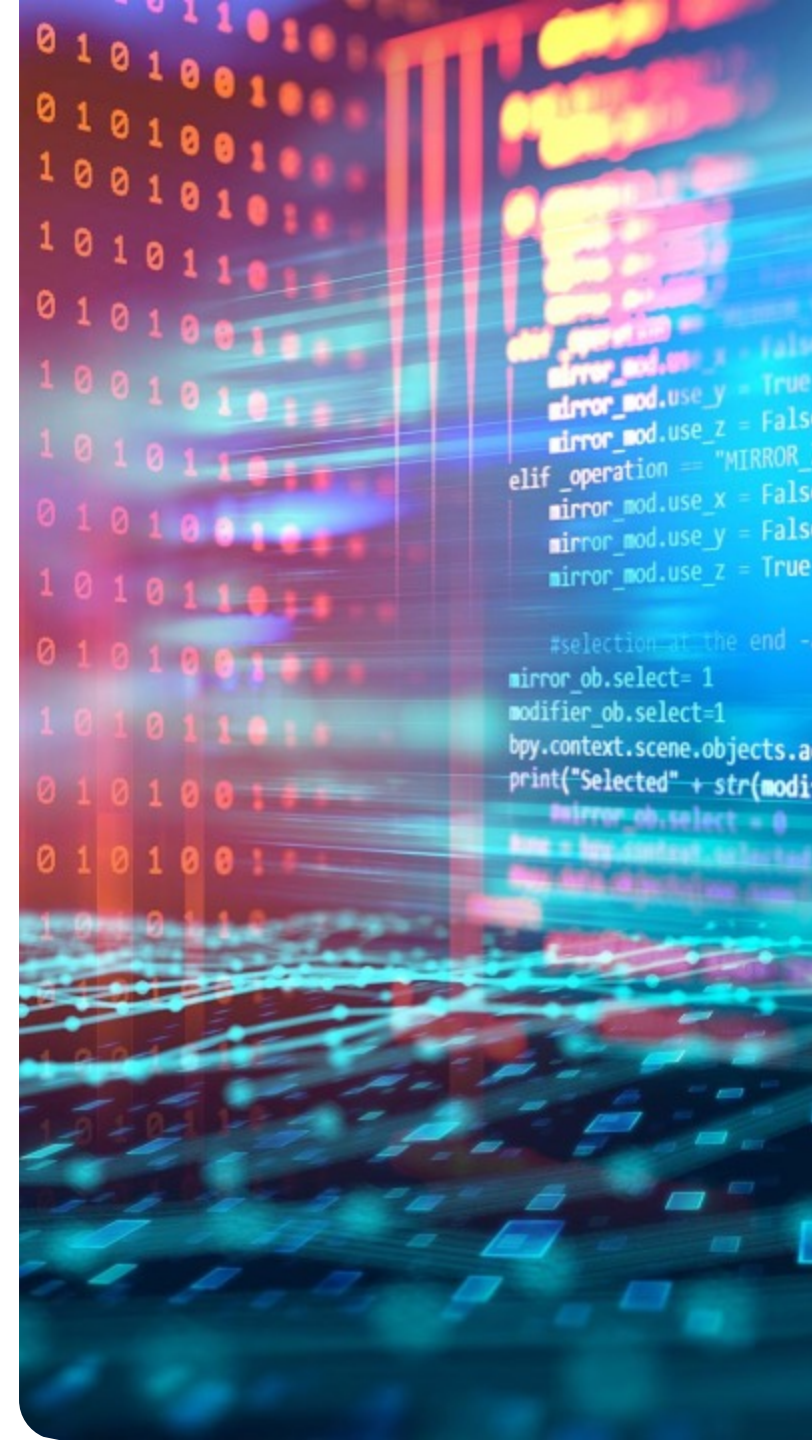


- ☐ implementuje oczekiwany interfejs,
- ☐ tłumaczy wywołania na interfejs klasy, którą chcemy wykorzystać.

Struktura



- ☐ **Target (interfejs docelowy)**
Interfejs oczekiwany przez klienta.
- ☐ **Adaptee (klasa adaptowana)**
Istniejąca klasa o niezgodnym interfejsie.
- ☐ **Adapter**
Klasa pośrednia, która implementuje interfejs Target i deleguje wywołania do Adaptee, przekształcając je w odpowiedni sposób.
- ☐ **Client (klient)**
Korzysta z obiektów poprzez interfejs Target, nie wiedząc o istnieniu klasy Adaptee.



Adapter

Zalety

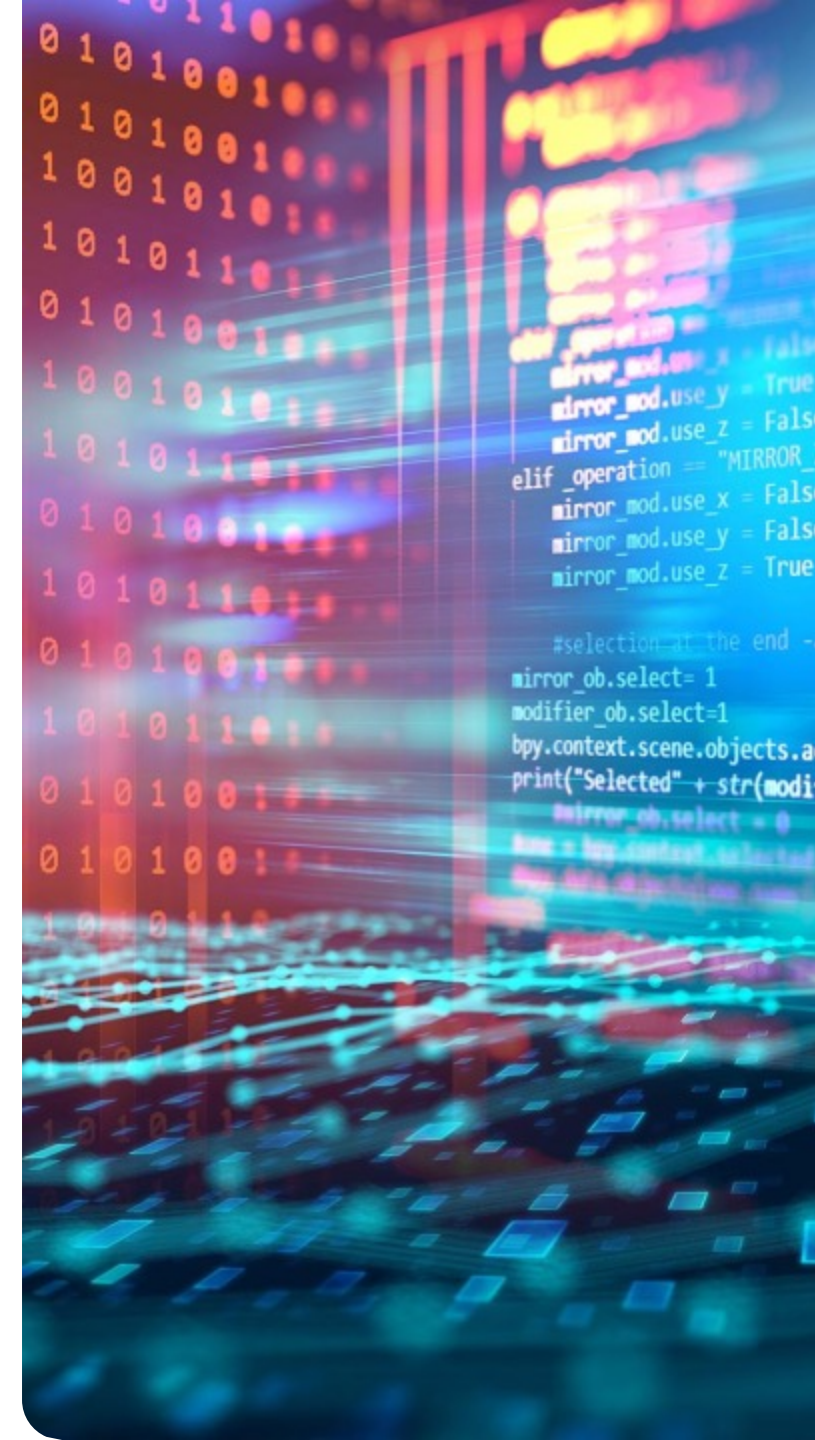
- **Możliwość integracji niekompatybilnych klas**
- **Ponowne wykorzystanie istniejącego kodu**
- Zmniejszenie zależności między komponentami
- Ułatwienie integracji z bibliotekami zewnętrznymi

Wady

- **Dodatkowa warstwa abstrakcji**
- Może zwiększać złożoność systemu
- W niektórych przypadkach wymaga tworzenia wielu adapterów

Przykłady zastosowania

- integracja z istniejącymi systemami lub bibliotekami,
- dostosowanie starego kodu (legacy) do nowych wymagań,
- łączenie komponentów o różnych interfejsach,
- budowa warstw pośrednich w systemach (np. API).







1. Czym są wzorce projektowe?



2. Jakie są główne grupy wzorców?



3. Jaka jest największa zaleta stosowania wzorców?



do your thing